

Spec-1: University AI Compute Management Platform

Juan Andrés Cano Ramírez — A00399560

Scope note: whole-system specification of governance, reservation, and quota behavior for the shared GPU fleet, per the brief's Iteration 1 request. Module decomposition, schema, UI, and transport technology stay open (see Non-Goals).

Contract

All ids below are opaque string identifiers with no assumed internal structure.

Entities. User: id, coarse role tier (student, instructor, researcher, operator, admin) deciding which actions are structurally possible, not which models/courses are reachable (fine-grained authorization stays undecided). AcademicContext (course/lab/group): id, token budget pool, GPU-hour budget pool, allowed models. WorkloadSubmission: user id, context id, model id, type (interactive inference, single-node training, distributed training), resource requirement (GPU count, minimum VRAM, worker count).

Submit workload. Input: user id, context id, model id, workload type, GPU count, minimum VRAM, worker count. Output: status (EXECUTING, QUEUED, PENDING_ADMIN_REVIEW, or a rejection); if queued, a position and an estimated start time; if rejected, exactly one reason: unauthorized model for context, exhausted student quota, exhausted course quota, hardware class absent from the fleet, or a distributed/high-VRAM job outside its allowed window; in every case, a quota snapshot of remaining student and course balance. Hardware that exists but is merely busy queues instead of rejecting. Quota enforcement is pre-emptive: an estimated token cap is reserved against both balances at admission, generation is truncated at that cap if exceeded, and the ledger reconciles to actual usage once the job completes.

Create reservation. Input: instructor id, resource requirement (GPU count, minimum VRAM), time window (start and end timestamp). Output: a granted reservation, an offered substitute (equivalent-or-better hardware free in that window), or PENDING_ADMIN_REVIEW if neither applies. When a granted window begins, unreserved jobs on that hardware receive a termination signal and a fixed grace period to checkpoint; a job still running when that period ends is forcibly terminated and its resources reclaimed, and is recorded as preempted rather than failed, so usage reports can tell the two apart. The exact rule for comparing hardware as "equivalent-or-better" is deliberately left undefined here: fixing that comparison would mean designing an internal scheduling algorithm, which this iteration is explicitly not meant to do (see Non-Goals); it is an intentional deferral, not an omission.

Check node status. Input: none, or an optional node id filter. Output: per node, one of available, reserved, busy, idle, or failing. **Check usage.** Input: a student id or a context id. Output: prompt and completion token consumption, plus GPU-time consumption, for that scope. Every reservation change, quota override, and administrative bypass logs actor, action, target, and timestamp.

Framing note: the fields, output shapes, and named error conditions above are binding; any transport shown (REST, gRPC, a queue) is illustrative only, not a stack commitment.

Why

The GPU fleet is shared across teaching, research, and coursework with no automated coordination: training jobs starve students needing the same hardware for a live lab, and nothing enforces how much compute a student can spend against a course budget. Instructors need their class hour guaranteed; students need clear denial reasons and queue estimates; researchers need to run large jobs without breaking someone else's class. This spec makes those guarantees enforceable by the system, not negotiated by hand.

Capabilities

- Submitting a workload returns running, queued with position, pending review, or a specific named rejection reason.
- A student's usage is checked against both personal and course quota before running; either exhausted blocks the request.
- An instructor can reserve hardware for a class window; unreserved jobs there are signaled to stop first.
- Overlapping reservations: earlier one honored, later one offered a substitute, escalating to a human only if none exists.
- Distributed/high-VRAM jobs run only in their designated window unless an admin explicitly overrides it.

- Any user can check live node status without infrastructure access.
- Any reservation, override, or quota change leaves a visible record of who, what, and when.

Constraints

The fleet is heterogeneous (24 GB single-GPU nodes plus one dual 48 GB node), so scheduling must account for mixed hardware, not assume uniformity; quota and reservation enforcement happens before a workload starts, not after; the authorization model (RBAC, ABAC, or hybrid) is not fixed and must not be assumed downstream; numeric thresholds (grace periods, timeouts) are tunable policy defaults, not fixed technical limits; any policy bypass must be logged, tied directly to Academic Administration's traceability goal.

Non-Goals

No internal module/microservice decomposition; no database schema or storage technology; no user interface or client application ("checking status" and "submitting a workload" describe behavior, not screens); no committed transport protocol (see Contract framing note); no physical provisioning or driver configuration; no fine-grained authorization model beyond context-level catalogs; no internal algorithm for computing hardware equivalence or scheduling heuristics (deliberately deferred, see Contract).

Success Signal

Every admitted class reservation gets exclusive hardware at its scheduled start with no interference from a running background job; a rejected workload always returns its specific named reason, never a generic failure; a workload needing merely-busy (not absent) hardware queues instead of rejecting, in 100% of cases; a conflicting later reservation always receives either a substitute offer or admin review, never a bare rejection; every reservation, override, and quota exception produces a matching, retrievable log entry.

Assumptions

- **Risky:** the LLM-serving layer reports exact prompt/completion token counts; if false, quota enforcement and metering have no reliable data and need an estimation fallback.
- **Risky:** this iteration assumes moderate concurrent load; a full-cohort simultaneous surge is not addressed and would need separate treatment.
- **Safe:** GPU hardware is exposed as schedulable units by the orchestration layer; without this the project has no foundation regardless of spec quality.
- **Safe:** cross-node clock sync is reliable enough for consistent ordering; nothing here needs sub-second precision.
- **Safe:** academic contexts and their membership already exist upstream; this spec governs usage within a context, not how contexts are created.